

Bilan de déploiement

- 1

create_tm1_process("Copy_Revenue_Units_v2", "24Retail")

✓

Processus créé — HasSecurityAccess: false · Caption: "Copy_Revenue_Units_v2"

Note : Copy_Revenue_Units existait déjà — non écrasé
- 2

update_tm1_process — tentative 1: IF(pUnitFcst <= 0) → Syntax error ligne 141

Cause : opérateur <= non supporté sur paramètre Numeric dans ce contexte
- 2

update_tm1_process — tentatives 2–4 : variables locales string/num dans IF → Syntax errors

Cause : ROUND(x, 0) invalide · variables locales string dans IF invalides
- 2

update_tm1_process — tentative 5 : INT(x) + IF/ELSE imbriqués

✓

syntax_errors: [] · Message: "Process updated successfully"

Paramètres du processus

#	PARAMÈTRE	TYPE	DÉFAUT	PROMPT
1	pOrg	String	100	Organisation (ex: 100)
2	pChannel	String	10	Canal (10=Retail, 20=Internet, 30=Distribution)
3	pCurrentProduct	String	21002	Code produit (ex: 21002)
4	pYear	String	Y1	Annee (ex: Y1)
5	pVersion	String	Forecast	Version cible (ex: Forecast)
6	pUnitFcst	Numeric	0	Total annuel a ventiler
7	pSpreadMethod	String	Uniform	Methode (Uniform ou Seasonal)

Clé saisonnière — Répartition réelle déployée

La clé [10,8,8,9,9,9,8,8,9,9,8,7] somme à **102** (pas 100). Le code normalise automatiquement : chaque clé = valeur brute ÷ 102. Dec absorbe les écarts d'arrondi.

MOIS	CLÉ BRUTE	% RÉEL	RÉPARTITION VISUELLE (BASE 1000 UNITÉS)
Jan	10	9.8%	98u
Fév	8	7.8%	78u
Mar	8	7.8%	78u
Avr	9	8.8%	88u
Mai	9	8.8%	88u
Jun	9	8.8%	88u
Jul	8	7.8%	78u
Aoû	8	7.8%	78u
Sep	9	8.8%	88u
Oct	9	8.8%	88u
Nov	8	7.8%	78u
Dec	7	6.9% + arrondi	70u + reste
Total	102	100%	1000u exactement ✓

```

# Validation paramètres string obligatoires
IF( TRIM(pOrg) @= ' ' );
    ProcessBreak();
ENDIF;
IF( TRIM(pVersion) @= ' ' );
    ProcessBreak();
ENDIF;

# Validation pSpreadMethod – IF/ELSE imbriqués obligatoires
# (@<> et variables locales string dans IF invalides sur ce serveur)
IF( pSpreadMethod @= 'Uniform' );
    vSpreadOk = 1;
ELSE;
    IF( pSpreadMethod @= 'Seasonal' );
        vSpreadOk = 1;
    ELSE;
        ProcessError();
        LogOutput( 'ERROR', 'pSpreadMethod invalide: ' | pSpreadMethod );
        ProcessBreak();
    ENDIF;
ENDIF;

# Blocage des versions calculées (8 versions réelles 24Retail)
IF( pVersion @= 'Variance' % pVersion @= 'Variance%'
    % pVersion @= 'Rate Variance' % pVersion @= 'Volume Variance'
    % pVersion @= 'Ccy Exchange Variance' % pVersion @= 'Performance Variance'
    % pVersion @= 'Target vs Prior Year Actual'
    % pVersion @= 'Target vs Prior Year Actual %' );
    ProcessError();
    LogOutput( 'ERROR', pVersion | ' est une version calculée.' );
    ProcessBreak();
ENDIF;

# Clé Uniform : 1/12 par mois, Dec = reste exact
IF( pSpreadMethod @= 'Uniform' );
    vKeyJan = 1 \ 12; vKeyFeb = 1 \ 12; vKeyMar = 1 \ 12;
    vKeyApr = 1 \ 12; vKeyMay = 1 \ 12; vKeyJun = 1 \ 12;
    vKeyJul = 1 \ 12; vKeyAug = 1 \ 12; vKeySep = 1 \ 12;
    vKeyOct = 1 \ 12; vKeyNov = 1 \ 12; vKeyDec = 1 - ( 11 \ 12 );
ENDIF;

# Clé Seasonal : normalisation sur total réel 102
IF( pSpreadMethod @= 'Seasonal' );
    vRawTotal = 10+8+8+9+9+9+8+8+9+9+8+7; # = 102
    vKeyJan = 10 \ vRawTotal; vKeyFeb = 8 \ vRawTotal;
    vKeyMar = 8 \ vRawTotal; vKeyApr = 9 \ vRawTotal;
    vKeyMay = 9 \ vRawTotal; vKeyJun = 9 \ vRawTotal;
    vKeyJul = 8 \ vRawTotal; vKeyAug = 8 \ vRawTotal;
    vKeySep = 9 \ vRawTotal; vKeyOct = 9 \ vRawTotal;
    vKeyNov = 8 \ vRawTotal; vKeyDec = 7 \ vRawTotal;
ENDIF;

# Calcul des unités – INT() obligatoire (ROUND() invalide sur ce serveur)
vUnitsJan = INT( pUnitFcst * vKeyJan );
vUnitsFeb = INT( pUnitFcst * vKeyFeb );
vUnitsMar = INT( pUnitFcst * vKeyMar );
vUnitsApr = INT( pUnitFcst * vKeyApr );
vUnitsMay = INT( pUnitFcst * vKeyMay );
vUnitsJun = INT( pUnitFcst * vKeyJun );
vUnitsJul = INT( pUnitFcst * vKeyJul );

```

```
vUnitsAug = INT( pUnitFcst * vKeyAug );
vUnitsSep = INT( pUnitFcst * vKeySep );
vUnitsOct = INT( pUnitFcst * vKeyOct );
vUnitsNov = INT( pUnitFcst * vKeyNov );
# Dec = reste pour garantir la somme exacte
vUnitsDec = pUnitFcst - vUnitsJan - vUnitsFeb - vUnitsMar - vUnitsApr - vUnitsMay - vUnitsJun - vUnitsJul
```

Code Epilog

validé syntaxiquement

EPILOG

```
# Écriture des 12 mois dans Revenue
# Ordre des dimensions (confirmé live) :
# organization, Channel, product, Month, Year, Version, Revenue
CellPutN( vUnitsJan, 'Revenue', pOrg, pChannel, pCurrentProduct, 'Jan', pYear, pVersion, 'Units Sold' );
CellPutN( vUnitsFeb, 'Revenue', pOrg, pChannel, pCurrentProduct, 'Feb', pYear, pVersion, 'Units Sold' );
# ... Mar → Nov identiques
CellPutN( vUnitsDec, 'Revenue', pOrg, pChannel, pCurrentProduct, 'Dec', pYear, pVersion, 'Units Sold' );
vSomme = vUnitsJan+vUnitsFeb+vUnitsMar+vUnitsApr+vUnitsMay+vUnitsJun+vUnitsJul+vUnitsAug+vUnitsSep+vUnitsOct
LogOutput( 'INFO', '12 mois charges. Total=' | NumberToString(vSomme) | ' Demande=' | NumberToString(pUnitFcst) );
LogOutput( 'INFO', 'Copy_Revenue_Units_v2: Termine.' );
```

Leçons de syntaxe TM1 — découvertes en déploiement live

PATTERN	STATUT	ALTERNATIVE CORRECTE
ROUND(x, 0)	❌ INVALIDE	INT(x)
IF(numParam <= 0)	❌ INVALIDE	IF(numParam = 0) ou logique ELSE
IF(localStringVar @= 'val')	❌ INVALIDE	IF directement sur le paramètre d'origine
pA @<> 'x' & pA @<> 'y'	❌ INVALIDE	IF/ELSE imbriqués sur @=
IF(pStr @= 'val'); ... ENDIF;	✅ VALIDE	—
IF(a % b % c) sur strings	✅ VALIDE	— (opérateur OR sur @= fonctionne)